HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

SCHOOL OF ELECTRICAL AND ELECTRONIC ENGINEERING

EMBEDDED CONTROL SYSTEMS DESIGN (EE4401E)

Topic: Design an Embedded System for DC Motor Control

Instructor:	**MS. Chu Đức Việt**
Students' name & ID:	Kiều Nguyễn Hoàng Thi - 20222800 Mai Xuân Thịnh – 20222866 Lưu Nguyễn Thành Trung – 20222868
Class ID:	161672

HUST, 04/02/2026

Design an Embedded System for DC Motor Control

I. Requirement Specification Document

1. General Information

- System: BLDC Motor, 24 VDC, 100 W, rated speed 105 rpm at shaft.
- Application:
  - Conveyor belt drive in small packaging/sorting workshops.
  - Typical load: Cartons up to 25 kg.
- Objective: Control and monitoring of Speed, Torque (estimated via current sensing), Rotation Direction, and System Faults.

2. Functional Requirements

2.1 Technical Specifications – Command Inputs (Digital Inputs - DI)

- General Purpose: Receive binary commands (ON/OFF) from push-buttons.
- Logic Thresholds:
  - Logic 0: $V_{IN} < 5$ V
  - Logic 1: $V_{IN} > 11$ V
- System Digital Inputs:
  - DI_1: Start/Stop:
    - 1: Enable drive and run according to speed setpoint.
    - 0: Stop.
  - DI_2: E-STOP (Emergency Stop):
    - Connected to a normally-closed (NC) emergency push-button.
    - Trigger: When the loop opens (cable broken/button pressed).
    - Action: Emergency stop; PWM/H-bridge is disabled.
  - DI_3: RESET:
    - Clears latched fault conditions (over-current, over-temperature, encoder error, etc.).
- Isolation Requirement: All DIs must be galvanically isolated from the MCU to reject 24V field noise, protect the 3.3V/5V MCU, and avoid damage from motor transients.

2.2 Status Outputs (Digital Outputs - DO)

- Logic Thresholds:
  - Logic 0: 0 V

    - Logic 1: 24 V
- System Digital Outputs:
    - DO_1 (FAULT / ALARM):
        - Normal: OFF (0 V).
        - Fault: ON (24 V) for shaft jam, overload, over-temperature, E-STOP, etc.
        - Usage: Drives a red alarm lamp.
    - DO_2 (RUNNING):
        - ON: Motor running (speed > threshold, no faults).
        - OFF: Stopped, fault state, or active E-STOP.
        - Usage: Drives a green lamp.
    - DO_3 (BRAKE CONTROL - Optional):
        - Controls electromagnetic/mechanical holding brake.
        - Logic: RUNNING → Brake released; STOP/E-STOP → Brake engaged.

2.3 Analog Inputs (AI)

- Purpose: Receive continuous-value references for speed setpoint or torque/current limits.
- AI_1 (SPEED REFERENCE):
    - Operating Range: 0–10 V ↔ 0–100% rated speed, or 4–20 mA ↔ 0–100% rated speed.
    - Front-end Circuitry: Voltage divider + Op-amp buffer (scale to 0–3.3V/5V), Zener diode clamp (3.3V/5V protection), and RC low-pass filter.
- AI_2 (TORQUE / CURRENT LIMIT - Optional):
    - Sets maximum allowed current. Same electrical interface as AI_1.

2.4 Analog Outputs (AO)

- Purpose: Provide analog monitoring signals (0–10 V or 4–20 mA).
- AO_1 (ACTUAL SPEED): 0 V (4 mA) → 0 rpm; 10 V (20 mA) → rated speed (105 rpm).
- AO_2 (CURRENT / TORQUE FEEDBACK): 0–10 V ↔ 0–I_max (e.g., 0–30 A).
- Output Circuitry: MCU PWM + RC low-pass filter + Op-amp buffer (0–10V amplification or voltage-to-current conversion).

2.5 Measurements and Control

- Speed Feedback: Incremental encoder (magnetic/optical), resolution ≥ 500 PPR, shaft-mounted.
- Current Sensing: Hall-effect sensor or Shunt resistor.
- Outputs: PWM signal driving H-bridge; Optional analog outputs for monitoring.
- Control Architecture:

- Speed Loop: PI controller, update period 5–10 ms (100–200 Hz).
- Inner Current Loop: PI controller, high bandwidth (several hundred Hz).
- Tuning: Kp, Ki gain adjustment via keypad or software.

2.6 Operating Modes & Communication

- **Modes:**
  - Speed Mode (Conveyor Regulation).
  - Stop (Standby).
  - Local (Keypad) / Remote (DI/AI or RS-485 Modbus).
- **Communication:**
  - Bus: RS-485, 2-wire, half-duplex, up to 115.2 kbps.
  - Protocol: Modbus RTU (Slave mode), configurable address.
  - Isolation: Optocoupler/transformer isolation ≥ 2.5 kVrms.

2.7 Protection Features

- Overvoltage / Undervoltage (24 VDC).
- Overcurrent, short-circuit, motor overload (I^2t), and stall detection.
- Overtemperature (Heatsink/Motor sensors).
- Reverse-polarity protection for 24 V supply.
- ESD protection (TVS diodes) and Galvanic isolation between control and power stages.

3. Non-functional Requirements

Category	Requirement
System Response	Accel: 0.5–5s (Def: 2s); Decel: Def 2s; E-STOP: ≤ 50 ms.
Speed Range	Motor shaft: 300–2000 rpm; Conveyor: 0–v_max m/s.
Update Time	Speed loop: 5–10 ms; Current/PWM loop: ≤ 1 ms.
Power Supply	24 VDC input.
Performance	Driver efficiency ≥ 95%; Overall efficiency ≥ 80–85%; Speed error: ±2%.
Environmental	Temp: –10°C to +50°C; Humidity: ≤ 90% RH; IP20/40 (Driver), IP44/54 (Motor).

4. User Interface

4.1 Display (16×4 LCD)

- Parameters: Speed (rpm/%), Current/Load (%), Direction, Status (RUN/STOP/FAULT).

4.2 Keypad & Interaction

- Keys: Run, Stop, Up, Down, Menu/Enter, Esc.
- Functions: Edit mode for setpoints, ramp times, Kp/Ki, Modbus settings, and password protection.

5. Safety Standards

- EMC/Safety: IEC 61000, IEC 60204-1.
- Functional Safety: IEC 61508, IEC 61131-2.
- General: Overvoltage, overcurrent, overtemperature, ESD, and MCU/IO Isolation.

6. Constraints

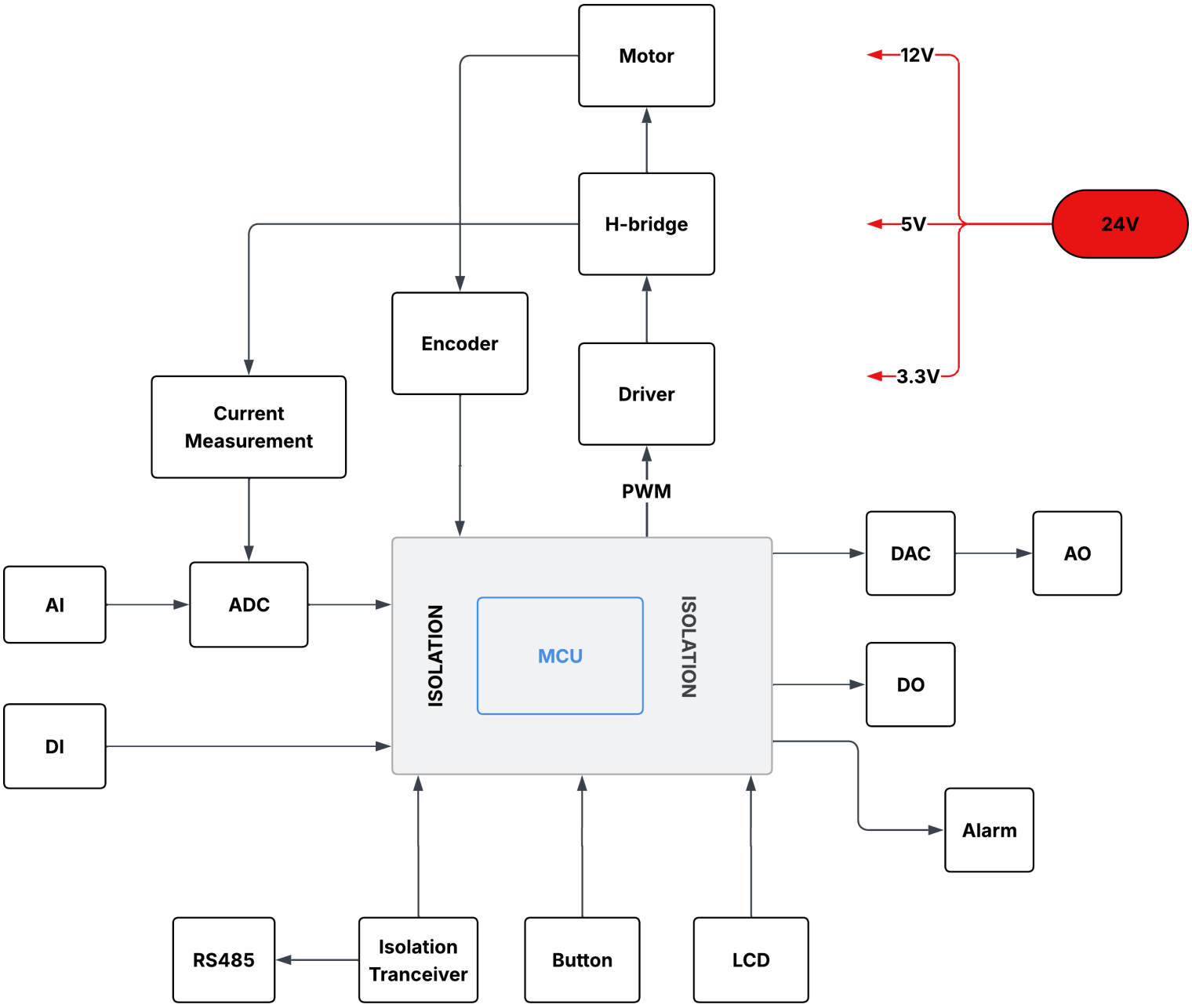
- Motor Type: Designed for brushed DC motors only (24V, 750W, S1 duty cycle).
- Power: Fixed nominal voltage (18–24 VDC) via AC-DC supply.
- Thermal: Requires heatsink + airflow; must be in a ventilated electrical cabinet.
- Connectivity: Supports RS-485/Modbus RTU only (no Wi-Fi/BLE built-in).
- Physical: PCB must be size-optimized for compact driver housing.

7. Application

- Conveyor Belt System.

II. System Design Document

1. Block Diagram Of The System



2. Hardware Design

Our Hardware Design consists of these parts:

2.1. Power Supply
2.2. Analog Input
2.3. Digital Input
2.4. Analog Output
2.5. Digital Output
2.6. Digital Output Type Relay (SSR)
2.7. Driver Circuit
2.8. H-Bridge
2.9. Communications
2.10. Encoder

2.11. Current Sensor
2.12. User Interface
2.13. MCU

2.1. Power Supply

Requirement:

- Suppy 24VDC.
- Reduce voltage to 12VDC, 5VDC, and 3.3VDC (3.3VDC is suitable for MCU and isolation).

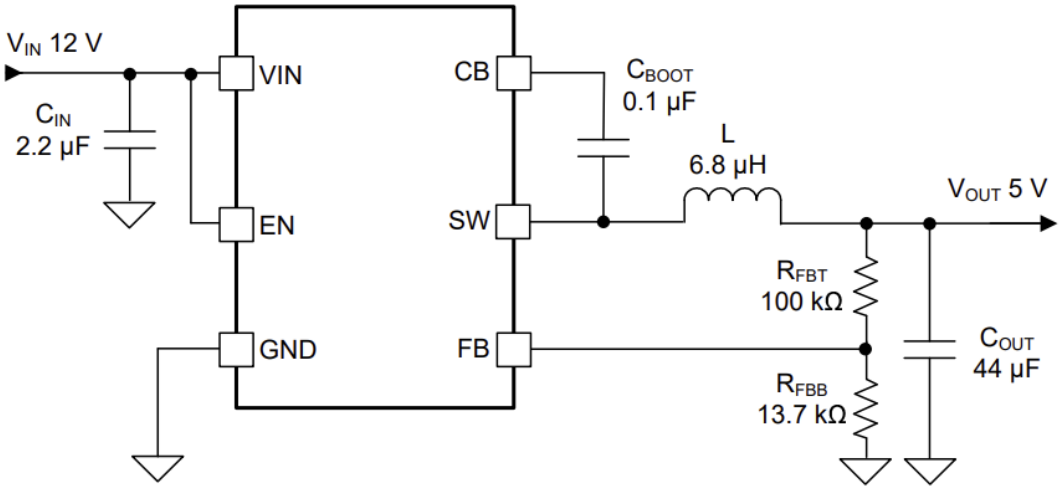
Hardware design:

Voltage source from 24V → 12V, 5V, 3.3V.

Device	LM68425-Q1	LMR51430	LM68645-Q1	LM2576S
$V_{in}(min)\ (V)$	3	4.5	3	4
$V_{in}(max)\ (V)$	42	36	65	40
$I_{out}(max)\ (V)$	2.5	3	4.5	3
$V_{out}(min)\ (V)$	0.8	0.6	0.8	3.3
$V_{out}(max)\ (V)$	40	34.2	4.5	37
Features	Fixed Output (5V, 3.3V)	Enable, Fixed soft start, Forced PWM, Light-load efficiency	Fixed Output (5V, 3.3V)	Fixed Output (12V, 5V, 3.3V)
Switching frequency (min) (kHz)	360	500	360	42
Switching frequency (max) (kHz)	2420	1100	2200	63
Operating temperature range (°C)	-40 to 150	-40 to 150	-40 to 150	-40 to 125
Price	$1.710	$0.563	$2.400	$1.535 (12V) $1.814 (5V, 3.3V)

The **LMR51430** is an easy-to-use synchronous step-down DC-DC converter operating from 4.5V to 36V supply voltage. The device is capable of delivering 3A DC load current in a very small solution size as the figure shown below.

The **LMR51430** employs fixed-frequency peak-current mode control and additional features such as precision enable and internal soft start provide a flexible abd easy-to-use solution for a wide range of applications (thermal shutdown, V_{IN} undervoltage lockout, cycle-by-cycle current limit, hiccup mode short-circuit protection).



Input voltage:

The EN pin provides electrical ON and OFF control for the LMR51430. When V_{EN} is below 0.95 V, the device is in shutdown mode. The LMR51430 also employs V_{IN} undervoltage lockout protection (UVLO). If V_{IN} voltage is below its UVLO threshold of 3.58 V, the converter is turned off.

Output Voltage Setpoint:

The output voltage of the LMR51430 device is externally adjustable using a resistor divider network. The divider network is comprised of a top feedback resistor R_{FBT} and bottom feedback resistor R_{FBB}. Equation (1) is used to determine the output voltage of the converter:

$$R_{FBT} = \frac{V_{out} - V_{REF}}{V_{REF}} \times R_{FBB} \qquad (1)$$

For example : 24V → 5V

Choose the value of R_{FBB} to be 13.7 kΩ. With the desired output voltage set to 5 V and the V_{REF} = 0.6 V, the R_{FBT} value can then be calculated using equation (1). The formula yields a value 100.4 kΩ. A standard value of 100 kΩ is selected as table 1.

Table 1. L and Cout Typical Values

f_{SW} (kHz)	V_{OUT} (V)	L (µH)	C_{OUT} (µF) (1)	R_{FBT} (kΩ)	R_{FBB} (kΩ)
500	3.3	5.6	2 × 22 µF / 25 V	100	22.1
	5	6.8	2 × 22 µF / 25 V	100	13.7
	12	12	3 × 22 µF / 25 V	100	5.23

Table 9-2. L and C_{OUT} Typical Values

f_{SW} (kHz)	V_{OUT} (V)	L (µH)	C_{OUT} (µF) (1)	R_{FBT} (kΩ)	R_{FBB} (kΩ)
1100	3.3	2.2	2 × 10 µF / 25 V	100	22.1
	5	3.3	2 × 10 µF / 25 V	100	13.7
	12	6.8	3 × 10 µF / 25 V	100	5.23

2.2. Analog Input

- Requirements

- Support one voltage input 0–10 V and one current input 4–20 mA.
- Protect inputs against ESD/surge and over-voltage.
- Filter noise to provide a stable DC signal for the ADC.
- Provide galvanic isolation between field signals and the MCU.

- Hardware design

For each analog input we use only:

- Protection network: TVS diode + series resistor + RC low-pass filter.
- External 12-bit, 4-channel SPI ADC MCP3204.
- Digital isolator: 4-channel isolator ADuM1401 for SPI signals.
- ADC:

2.2.1. Voltage input 0 - 10 V (prefered)

a) Protection and filtering

- TVS diode: SMBJ11CA connected between the signal line and isolated ground (GND_ISO) to clamp ESD and surge voltages above ~11 V.
- Input RC filter: capacitor $C_{IN} = 100$ nF and resistor $R_{IN} = 10$ kΩ from the protected node to GND, forming a low-pass filter with corner frequency suitable for slow industrial signals.

b) Scaling network

- To keep the ADC input in range of 0 - 3.3V and below 5.6 V, the 0 - 10 V signal is attenuated using a simple resistor divider and the output is routed to CH0 of MCP3204.
- $R_1 = 10$ kΩ from input node to divider midpoint.
- R_2 from midpoint to GND_ISO.

The voltage at the divider midpoint is:

$$V_{ADC} = V_{IN} \cdot \frac{R_2}{R_1 + R_2 + R_{IN}} = V_{IN} \cdot \frac{R_2}{10 + R_2 + 10} \qquad \boxed{(1)}$$

- <u>Condition 1</u>: ADC range 0 - 3.3V → With $V_{IN} = 10$ V, $V_{ADC} = 3.3$ V, → from equation (1) we have : $3.3(V) = 10(V) \cdot \frac{R_2}{10+R_2+10}$

  → $R_2 = 9.85$ kΩ

- <u>Condition 2</u>: ADC input below 5.6V (In order to protect MCP3204), we can see that

$$5.6(V) > V_{IN_max} \cdot \frac{R_2}{10 + R_2 + 10} = 13.5(V) \cdot \frac{R_2}{10 + R_2 + 10} \quad \rightarrow R_2 < 14.18\text{k}\Omega$$

So choose $R_2 = 9.85$ kΩ

c) ADC interface.

MCP3204: Which is a 12-bit SAR (Successive Approximation Register) analog-to-digital converter from Microchip with four multiplexed analog input channels (CH0–CH3) and an SPI-compatible serial interface. It includes an internal input multiplexer and sample-and-hold circuit, and supports single-ended or pseudo-differential measurements depending on the command

sent over SPI. The conversion range is set by the reference voltage (V_{REF}) (single-ended inputs are specified from V_{SS} to V_{REF}), and V_{REF} can be chosen from 0.25·V_{DD} up to V_{DD}.

$$\text{Digital Output Code} = \frac{4096 \times V_{IN}}{V_{REF}}$$

We choose $V_{DD} = 5V\ and\ V_{REF} = 3.3V$.

2.2.2. Current input 4–20 mA

a) Protection and filtering

- The MAX14626 current loop protector features a current-limit switch to prevent damage to the sensor devices due to faulty current loop conditions. The current-limit switch features a 25Ω (typ) on-resistance and operates from a +2.3V to +36V input voltage range. The accurate current limit is set to 30mA, making the part ideal for protecting the sensor reading devices. The MAX14626 handles an overcurrent event in a continuous current-limit mode. Additional safety features include thermal shutdown to prevent overheating and reverse input blocking to protect from being reverse connected into the sensor.
- A small series resistor $R_{IN} = 100\Omega$ plus a capacitor $C_{IN2} = 1u$F across R_{SHUNT} form a low-pass filter to suppress high-frequency noise on the loop.

b) Current-to-voltage conversion

- The 4–20 mA input uses a precision shunt resistor: $R_{SHUNT} = 165\Omega$
- The sensor current flows through R_{SHUNT}, generating a voltage:

$$V_{SHUNT} = I_{IN} \cdot R_{SHUNT}$$

  - At 4 mA: $V_{SHUNT} = 0.66$ V
  - At 20 mA: $V_{SHUNT} = 3.3$ V
- Output is routed to CH1 of MCP3204.

So the loop current is converted directly to a 0.66–3.3 V signal, perfectly matched to the ADC range.

2.2.3. ADC and isolation

a) ADC (MCP3204)

- 12-bit, 4-channel SPI ADC powered from isolated 5 V.
- CH0: 0–10 V input after scaling (0-3.3V).
- CH1: 4–20 mA input after shunt (0.66-3.3V)
- CH2/CH3: reserved for future analog inputs (e.g. more sensors).

b) Digital isolation (ADuM1401)

To isolate the MCU from the analog domain:

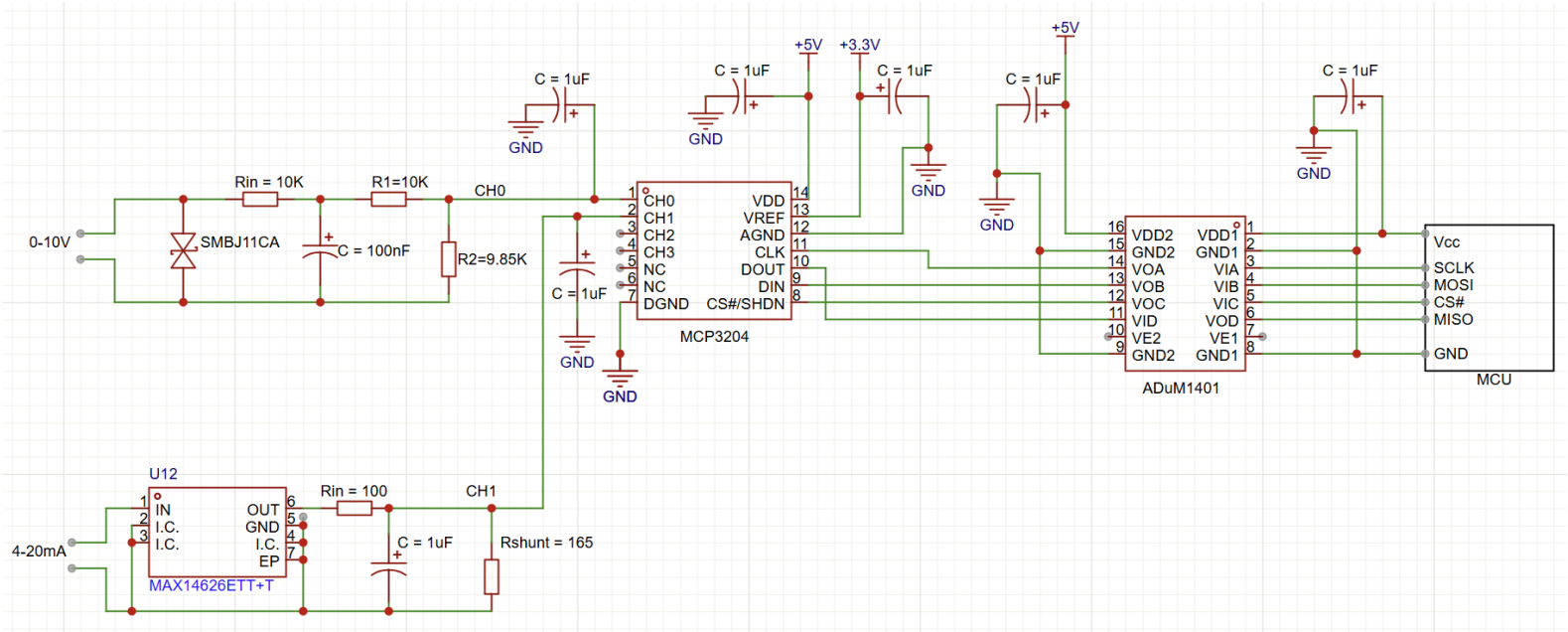
- ADuM1401 provides four isolated channels for SPI:
  - SCLK, MOSI and CS from MCU → ADC.

- MISO from ADC → MCU.

- Side 1 (MCU side): powered by 3.3 V or 5 V MCU supply and referenced to GND_MCU.
- Side 2 (analog side): powered by 5 V isolated rail and referenced to GND_ISO.

- CLK (SCLK) – clock input
- DIN (MOSI – data to ADC)
- DOUT (MISO – data from ADC)
- CS (chip select, active low)

Full schematic:



2.3. Digital Input

- Requirements

The digital input must:

- Accept industrial logic levels according to the project specification:
  - Logic LOW: 0–5 V
  - Logic HIGH: 11–30 V.
- Be compatible with both PNP and NPN sensors.
- Convert the field input to a 0–3.3 V logic level suitable for the MCU.
- Protect the MCU against ESD, over-voltage and negative voltage conditions, and provide isolation between field side and controller side.

- Design

60-V Zener diode: Protect against input overvoltage by clamping the field-side node (IN/SENSE side) during fast transients or when VIN can exceed 60 V, so the voltage at the ISO121x pins stays below 60 V.

ISO1211: The ISO1211 device is fully-integrated, isolated digital-input receivers with IEC 61131-2 Type 1, 2, and 3 characteristics. This devices is suitable for motor control digital input modules. The devices receive 24V to 60V digital input signals and provide isolated digital outputs. No field side power supply is required. An external resistor, R_{SENSE}, on the input signal path precisely sets the limit for the current drawn from the field input. This current limit helps minimize power dissipated in the system. The voltage transition thresholds can be increased further using an external resistor, R_{THR}. For more information on selecting the R_{SENSE} and R_{THR} resistor values, see the equation (3) and table 3 . The ISO1211 device is capable of high speed operation and can pass through a minimum pulse width of 150 ns. The ISO1211 device has a single receive channel.

The output buffers on the control side are capable of providing enough current to drive status LEDs as shown in table 2. and figure below:

V_{OH}	High-level output voltage on OUTx	V_{CC1} = 4.5 V; I_{OH} = –4 mA V_{CC1} = 3 V; I_{OH} = –3 mA V_{CC1}= 2.25 V; I_{OH} = –2 mA, see Figure 10	V_{CC1} – 0.4		V
V_{OL}	Low-level output voltage on OUTx	V_{CC1} = 4.5 V; I_{OH} = 4 mA V_{CC1} = 3 V; I_{OH} = 3 mA V_{CC1}= 2.25 V ; I_{OH} = 2 mA, see Figure 10		0.4	V

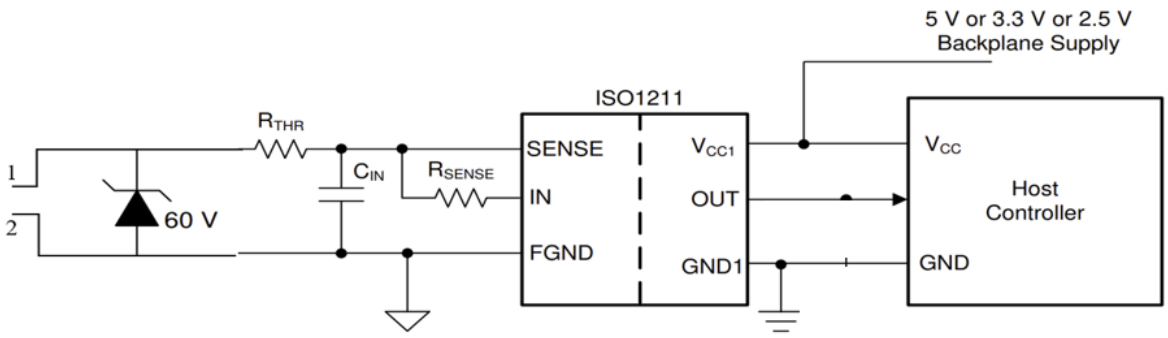


Table 2. Logic I/O

Table 3. Voltage transition threshold on field side.

PARAMETER		TEST CONDITIONS	MIN	TYP	MAX	UNIT
VOLTAGE TRANSITION THRESHOLD ON FIELD SIDE						
V_{IL}	Low level threshold voltage at module input (including R_{THR}) for output low	R_{SENSE} = 562 Ω, R_{THR} = 0 Ω, see Figure 11	6.5	7		V
		R_{SENSE} = 562 Ω, R_{THR} = 1 kΩ, see Figure 11	8.7	9.2		
		R_{SENSE} = 562 Ω, R_{THR} = 4 kΩ, see Figure 11	15.2	15.8		
V_{IH}	High level threshold voltage at module input (including R_{THR}) for output high	R_{SENSE} = 562 Ω, R_{THR} = 0 Ω, see Figure 11		8.2	8.55	V
		R_{SENSE} = 562 Ω, R_{THR} = 1 kΩ, see Figure 11		10.4	10.95	
		R_{SENSE} = 562 Ω, R_{THR} = 4 kΩ, see Figure 11		17	18.25	

$$V_{IH}(typ) = 8.25(V) + R_{THR} \times \frac{2.25mA \times 562\Omega}{R_{SENSE}}$$

$$V_{IL}(typ) = 7.1(V) + R_{THR} \times \frac{2.25mA \times 562\Omega}{R_{SENSE}}$$

So we choose:

$R_{SENSE} = 562\Omega$, $R_{THR} = 1k\Omega$, $C_{IN} = 1nF$

$$=> \begin{cases} V_{IL_min} = 8.7V \ and \ V_{IL_typ} = 9.2V \\ V_{IH_typ} = 10.4V \ and \ V_{IL_max} = 10.95V \end{cases}$$

Which means:

With $V_{IN} = 0 - 5(V) < 8.7(V)$: $this\ is\ low$

With $V_{IN} = 11 - 30(V) > 10.95(V)$: $this\ is\ high$

2.4. Analog Output.

- Requirement:

- 2 types of Analog signal: Current Analog signal 4-20mA, Voltage Analog signal 0-10V.
- Isolate AO signals between systems or protect the MCU from voltage spikes from pheripherals.
- Can protect high voltage, low voltage and static electicity.

- Hardware design:

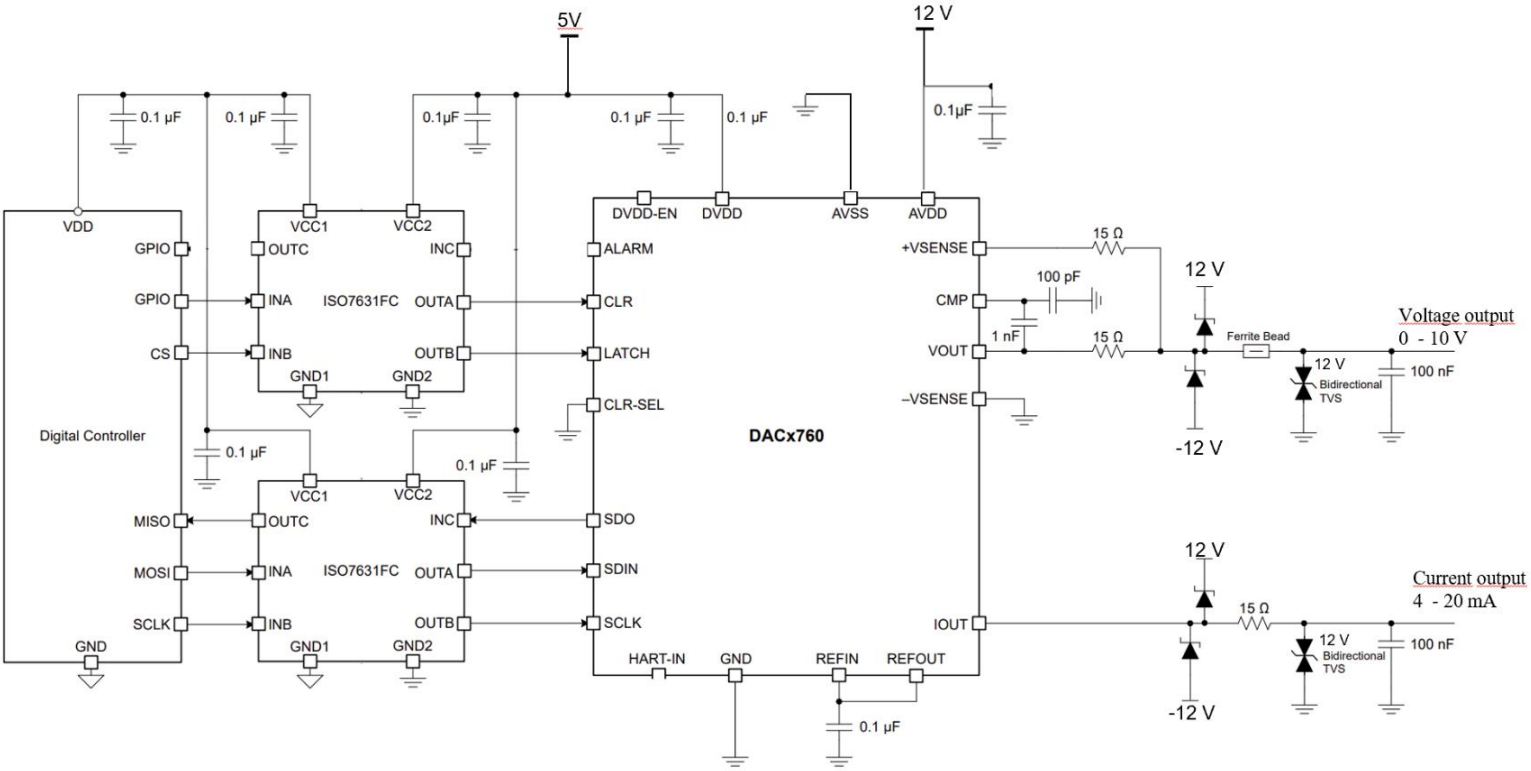
ISO7631FC is a three-channel digital isolator designed to provide galvanic isolation between two parts of a system while still allowing fast digital signals to pass across the isolation barrier. It is commonly used to isolate communication and control lines such as SPI-style signals (clock, data, latch/clear) between a low-voltage MCU domain and a higher-voltage or noisy field domain. With two forward channels and one reverse channel, the device can transmit control signals in one direction and return status or fault signals in the other, helping improve noise immunity, ground-loop protection, and system safety in industrial and power-electronics applications.

DAC7760 is a 12-bit, SPI-programmable digital-to-analog converter intended for industrial analog outputs, providing both a voltage output (programmable ranges including 0–5 V, 0–10 V, ±5 V, ±10 V) and a current output (programmable 4–20 mA, 0–20 mA, 0–24 mA). It integrates a 5 V internal reference (REFOUT) and includes power-on reset that starts the device in a known safe state where both VOUT and IOUT are disabled (Hi-Z). The device is controlled through a 24-bit SPI frame (8-bit address + 16-bit data) and uses a LATCH rising edge to load registers and update outputs. It also supports a dual-output mode where voltage and current outputs can be enabled simultaneously while being driven by a single data register, useful for flexible analog output configurations.

Input:

PARAMETER	TEST CONDITIONS	MIN	TYP	MAX	UNIT
VOLTAGE OUTPUT					
Voltage output ranges (normal mode)	AVDD ≥ 10 V	0		5	V
	AVDD ≥ 10.5 V	0		10	
	AVSS ≤ –5.5 V, AVDD ≥ 10 V	–5		5	
	AVSS ≤ –10.5 V, AVDD ≥ 10.5 V	–10		10	
Voltage output range (overrange mode)	AVDD ≥ 10 V	0		5.5	V
	AVDD ≥ 11.5 V	0		11	
	AVSS ≤ –6 V, AVDD ≥ 10 V	–5.5		5.5	
	AVSS ≤ –11.5 V, AVDD ≥ 11.5 V	–11		11	
Resolution	DAC8760	16			Bits
	DAC7760	12			

We choose: AVDD = 12V; DVDD = 5V.



With output voltage: 0 – 10V, we have:

$$V_{OUT} = V_{REF} \times GAIN \times \frac{CODE}{2^N}$$

Where:

- CODE is the decimal equivalent of the code loaded to the DAC
- N is the bits of resolution; and 12 for DAC7760
- V_{REF} is the reference voltage; for internal reference, V_{REF} = 5 V
- GAIN is automatically selected for a desired voltage output range as shown in this table:

VOLTAGE OUTPUT	GAIN
0 V to 5 V	1
0 V to 10 V	2
±5 V	2
±10 V	4

The voltage range is set according to the value of the RANGE bits and the OVR bit in the Control Register.

With current output: 4 – 20mA, we have:

$$I_{OUT} = 16mA \times \frac{CODE}{2^N} + 4mA$$

The current-output range is normally set according to the value of the RANGE bits in the Control Register. When both the voltage and current outputs are enabled in dual-output mode, the range is set by the IOUT RANGE bits in the Configuration Register.

Implement on software: in order to have 0-10V voltage ouput and 4-20mA current output.

Case 1: Enable both V_{OUT} (0–10 V) and I_{OUT} (4–20 mA)

This is done in firmware in the following order:

1. Write Configuration Register (0x57): set DUAL OUTEN = 1, set IOUT RANGE = 01 (4–20 mA).
2. Write Control Register (0x55): set OUTEN = 1, set RANGE = 001 (0–10 V).
3. Write DAC Data Register (0x01): write the output code → both VOUT and IOUT will change together according to the same DAC code.

Case 2: Enable only one output at a time.

1. Write Configuration Register (0x57): set DUAL OUTEN = 0.
2. Write Control Register (0x55): set OUTEN = 1.
    - To output 0–10 V: set RANGE = 001.
    - To output 4–20 mA: set RANGE = 101.

2.5. Digital Output

- Requirement:

- Digital output signal LOW: 0V, HIGH: 24V
- Isolate with MCU.
- Can protect high voltage, low voltage and static electricity ESD.

- Hardware Design:

- AQY212SZ is a PhotoMOS (solid-state relay) in a compact SOP-4 package that provides galvanic isolation between the control input and the load terminals. It uses an LED-driven MOSFET output (back-to-back MOSFETs) to behave like a normally-open switch for AC or DC loads, with low leakage when off and a low on-resistance when on. It is rated for up to 60 V load voltage and can switch up to 0.5 A continuous load current, making it suitable for isolated 24 V digital outputs such as driving indicator lamps or buzzers.

Item			Symbol	AQY212S	AQY210S	AQY214S
Input	LED operate current	Typical	I_{Fon}	0.9 mA		
		Maximum		3 mA		
	LED turn off current	Minimum	I_{Foff}	0.4 mA		
		Typical		0.85 mA		
	LED dropout voltage	Typical	V_F	1.25 V (1.14 V at I_F = 5 mA)		
		Maximum		1.5 V		
Output	On resistance	Typical	R_{on}	0.83 Ω	17 Ω	25 Ω
		Maximum		2.5 Ω	25 Ω	35 Ω
	Off state leakage current	Maximum	I_{Leak}	1 μA		
Transfer characteristics	Turn on time*	Typical	T_{on}	0.65 ms	0.23 ms	0.21 ms
		Maximum		2 ms	0.5 ms	0.5 ms
	Turn off time*	Typical	T_{off}	0.08 ms	0.04 ms	
		Maximum		0.2 ms		
	I/O capacitance	Maximum	C_{iso}	1.5 pF		
	Initial I/O isolation resistance	Minimum	R_{iso}	1,000 MΩ		

The maximum current for LED operate : $I_{Fon} = 3mA$

The typical current for LED turn off : $I_{Foff} = 0.85mA$

Voltage drop on Led 1.25-1.5V, voltage of high level of MCU is 3.3V so we have

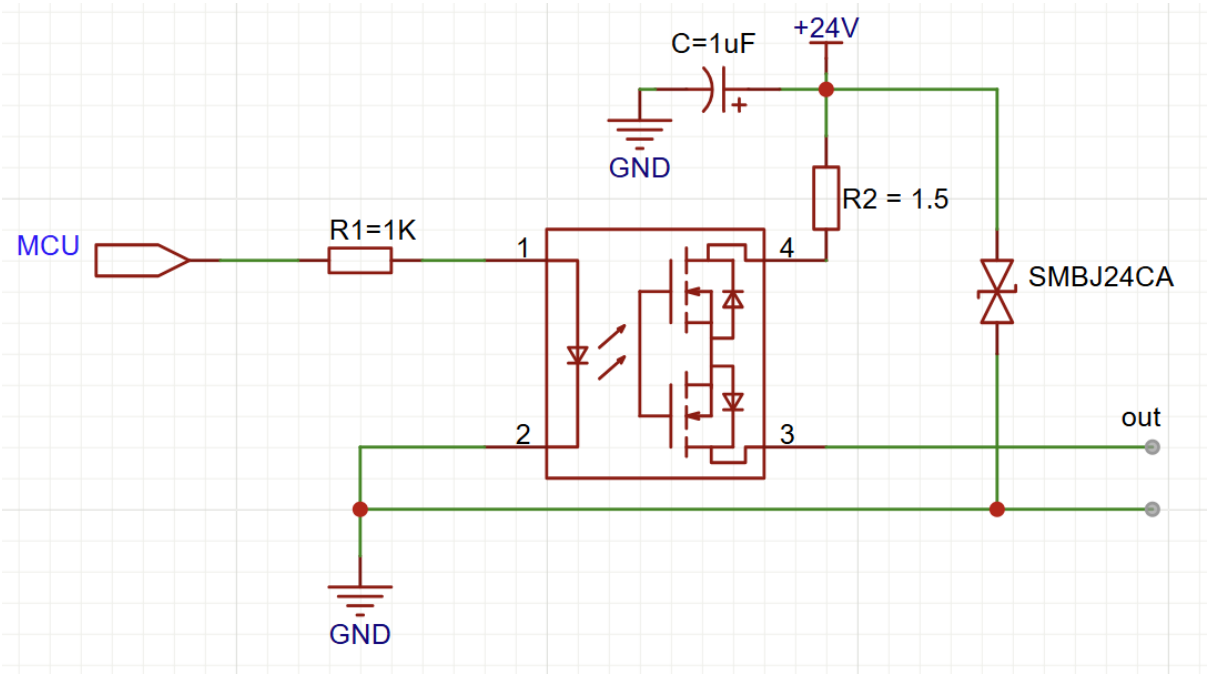
$$0.85(mA) < I_{LED} = \frac{V_{MCU} - V_{LED}}{R_1} < 3(mA)$$

$$0.6k\Omega < R_1 < 2.12k\Omega$$

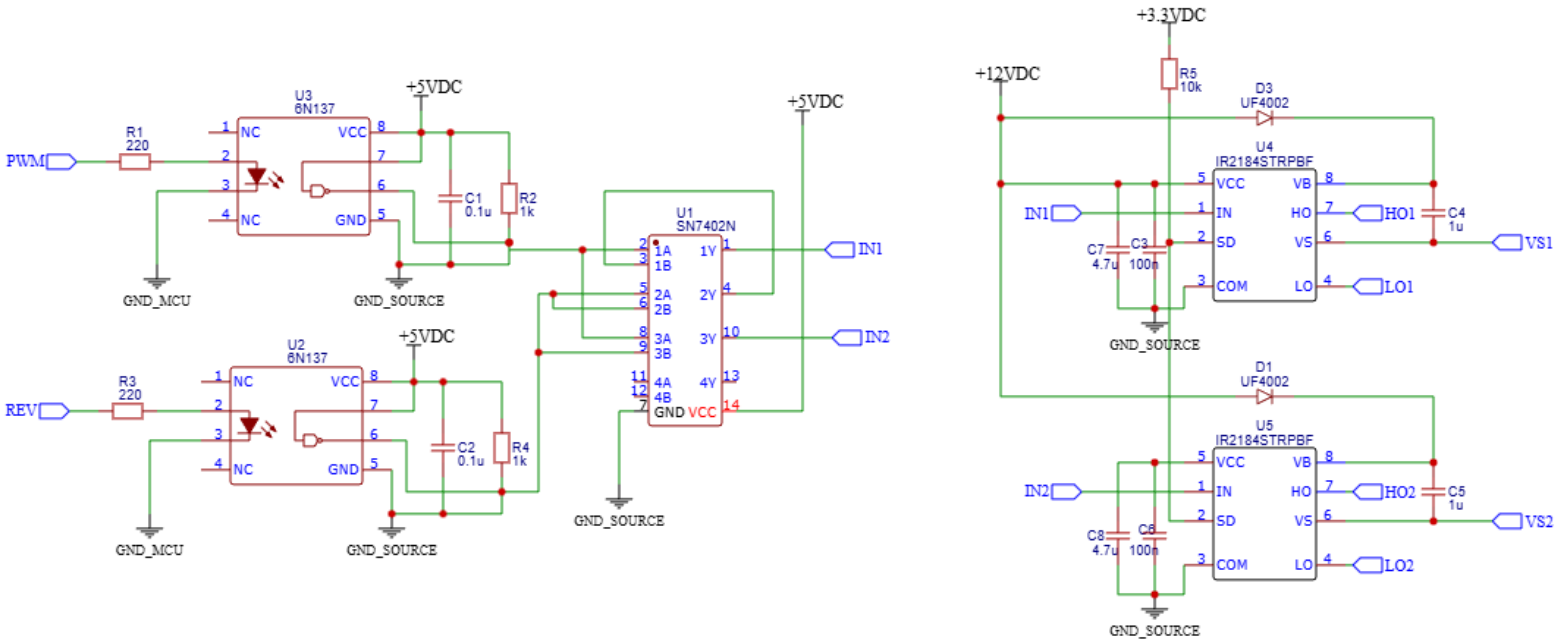
So we choose R1 = 1 kΩ.

According to the table, we choose R2 = 1.5Ω

- SMBJ24CA: Which is a bidirectional TVS diode (SMB / DO-214AA) used to protect 24-V rails and I/O lines from ESD and surge spikes by clamping transient overvoltage events.



2.6 Driver Circuit for H Bridge



2.6 Driver Circuit for H-Bridge

2.6.1 Requirements

- Galvanic isolation is preferred between GND_MCU and GND_SOURCE to improve noise immunity against high di/dt switching transients in the power stage.
- Power MOSFETs shall be driven with sufficiently high V_{GS} to minimize conduction losses.
- The gate-drive stage shall mitigate shoot-through via enforced dead-time and robust operation under switching-node transients.

2.6.2 Design

- Input isolation: 6N137 optocouplers galvanically isolate MCU control signals (PWM and direction/REV as applicable) from the power ground (GND_MCU ↔ GND_SOURCE) and translate them into logic referenced to GND_SOURCE.

- Logic conditioning/combining: SN7402N (quad NOR) is used as a logic interlock to generate IN1 and IN2 such that they are complementary (opposite states). This ensures the two half-bridge commands never conflict, preventing invalid drive combinations and reducing the risk of shoot-through during direction/PWM switching.
- Gate drive: Two IR2184S half-bridge drivers (VCC = 12 V) generate HO/LO gate signals for the H-bridge at 20 kHz PWM, leveraging the device's internal ~500 ns dead-time to reduce shoot-through risk.
- High-side supply: A bootstrap network (UF4002 + 1 μF) provides the floating high-side supply (VB–VS) for each half-bridge.

2.6.3 Component Calculation and Selection

a) LED series resistor selection (R1, R3)

The 6N137 input LED requires a forward current on the order of a few mA for reliable switching. For a 3.3 V MCU output, the design targets $I_F \approx 8$mA. Using a practical LED forward voltage $V_F \approx 1.4$V, the series resistor is:

$$R \approx \frac{V_{MCU} - V_F}{I_F} = \frac{3.3 - 1.4}{0.008} \approx 237\ \Omega$$

Therefore, R1 = R3 = 220 Ω is selected to provide adequate LED-drive margin.

b) Pull-up resistor selection (R2, R4)

The 6N137 output stage is used with a pull-up resistor to form a valid logic output referenced to +5 V. With R2 = R4 = 1 kΩ, the pull-up current when the output is LOW is approximately:

$$I \approx \frac{5}{1k} = 5\ \text{mA}$$

This current is within the 6N137 sink capability while providing fast rise times and robust logic levels.

c) Local bypass capacitors (C1, C2)

A 0.1 μF (100 nF) MLCC is placed close to each 6N137 VCC–GND pair to suppress local switching transients and ensure stable operation.

2.6.4 Half-bridge gate driver selection (IR2184S)

The IR2184S is selected because it:

- supports bootstrap high-side driving (VB–VS) for N-MOSFET half-bridges,
- provides strong gate-drive capability suitable for power MOSFET switching,
- operates from a 12 V gate-drive supply with low $R_{DS(on)}$ and
- includes internal ~500 ns dead-time to reduce cross-conduction (shoot-through).

2.6.5 Supply decoupling at IR2184 (C3/C6 and C7/C8)

For each driver, 100 nF (C3/C6) is placed in parallel with 4.7 μF (C7/C8) across VCC–COM:

- 100 nF: supplies high-frequency transient current during fast gate transitions.

- 4.7 µF: provides local energy storage to prevent VCC droop during repetitive switching bursts.

2.6.6 Bootstrap network (D3/D1 and C4/C5)

a) Bootstrap capacitor (C4/C5 = 1 µF)

The bootstrap capacitor stores energy to create the floating high-side supply. It charges when VS is low (low-side conduction) through the bootstrap diode, and during high-side operation it maintains:

$$V_{BS} = V_B - V_S$$

allowing the driver to produce:

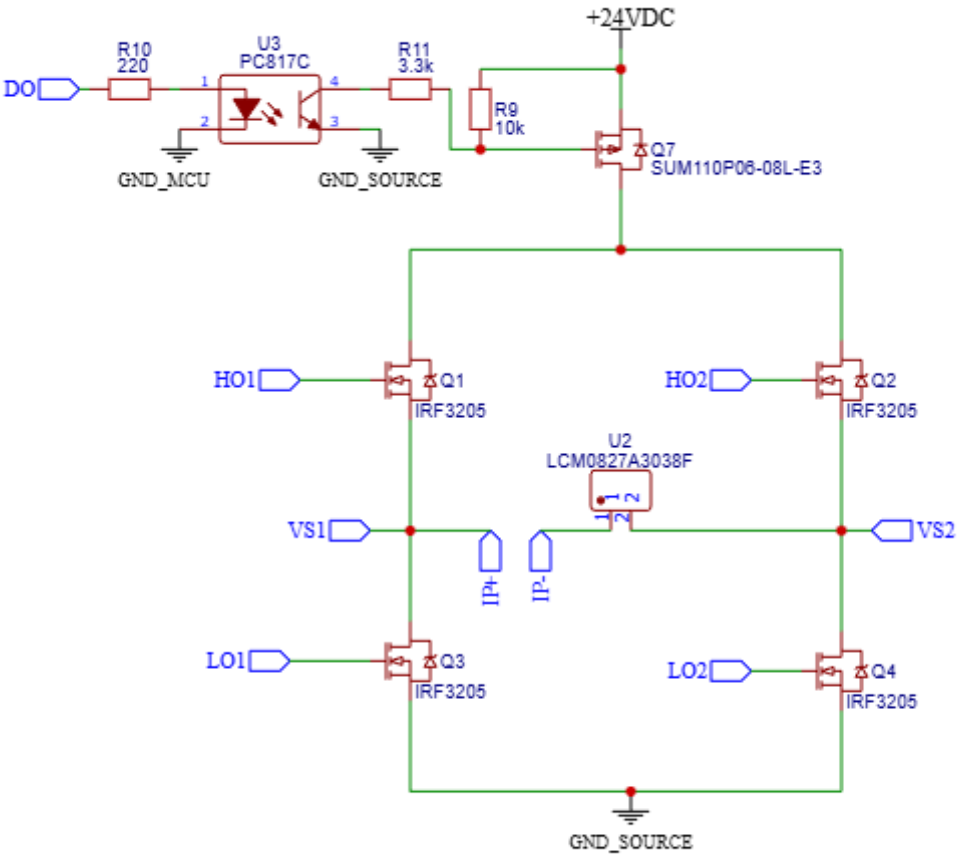
$$V_{HO} \approx V_S + V_{BS}$$

thus ensuring adequate high-side V_{GS}.

b) Bootstrap diode (D1/D3 = UF4002)

UF4002 is an ultrafast recovery diode used to charge $C_{BOOT}$when VS is low and to block reverse current when VS rises. Its fast reverse-recovery behavior is suitable for PWM switching environments.

2.7 H Bridge



Requirements

- Bus/load: $V_{bus} = 24\,\mathrm{VDC}$, motor power $P = 250\,\mathrm{W} \Rightarrow I_{nom} = P/V_{bus} = 250/24 \approx 10.4\,\mathrm{A}$. With a 1.5× design margin, $I_{pk} \approx 15.6\,\mathrm{A}$.
- Voltage derating: Power MOSFETs shall satisfy $V_{DS} \geq 1.5\,V_{bus} = 36\,\mathrm{V}$to tolerate ripple and transient overvoltage.
- Functional safety: Maintain galvanic isolation between MCU and power domains, and provide a hardware mechanism to disable the power stage.

Design

- H-bridge power stage: The motor is driven by a full N-MOSFET H-bridge (Q1–Q4). The switching nodes VS1 and VS2 connect to the motor terminals, enabling bidirectional operation, dynamic braking, and PWM torque/speed control.
- Global bus enable/disconnect: A series high-side bus switch (Q7) is inserted on the +24 V rail to provide a system-level power enable/shut-off for the entire bridge.
- Isolated enable control: Q7 is actuated via an opto-isolated control path (U3 = PC817C), preserving isolation between GND_MCU and GND_SOURCE.

2.7.1 Component Calculation & Selection

a) H-bridge MOSFETs (Q1–Q4 = IRF3205)

IRF3205 is selected because:

- $V_{DS} = 55\,\mathrm{V}$ exceeds the derating target $36\,\mathrm{V}$.
- $V_{GS,\max} = \pm 20\,\mathrm{V}$ is compatible with the intended $\sim 12\,\mathrm{V}$gate drive (with appropriate overshoot control).

b) Optocoupler LED drive (U3, R10)

With $R10 = 220\,\Omega$, $V_{DO} = 3.3\,\mathrm{V}$, and $V_F \approx 1.2\,\mathrm{V}$, the LED current is:

$$I_F = \frac{V_{DO} - V_F}{R10} \approx \frac{3.3 - 1.2}{220} \approx 9.6\,\mathrm{mA}$$

This current level is compatible with typical MCU GPIO drive capability and ensures reliable optocoupler operation.

c) Q7 gate bias and pull-down network (R9, R11)

- R9 (10 kΩ): pulls the Q7 gate to +24 V, keeping Q7 OFF by default (failsafe).
- R11 (3.3 kΩ): limits the optocoupler pull-down current to the gate, reducing stress on U3 and controlling the gate discharge rate during turn-on.

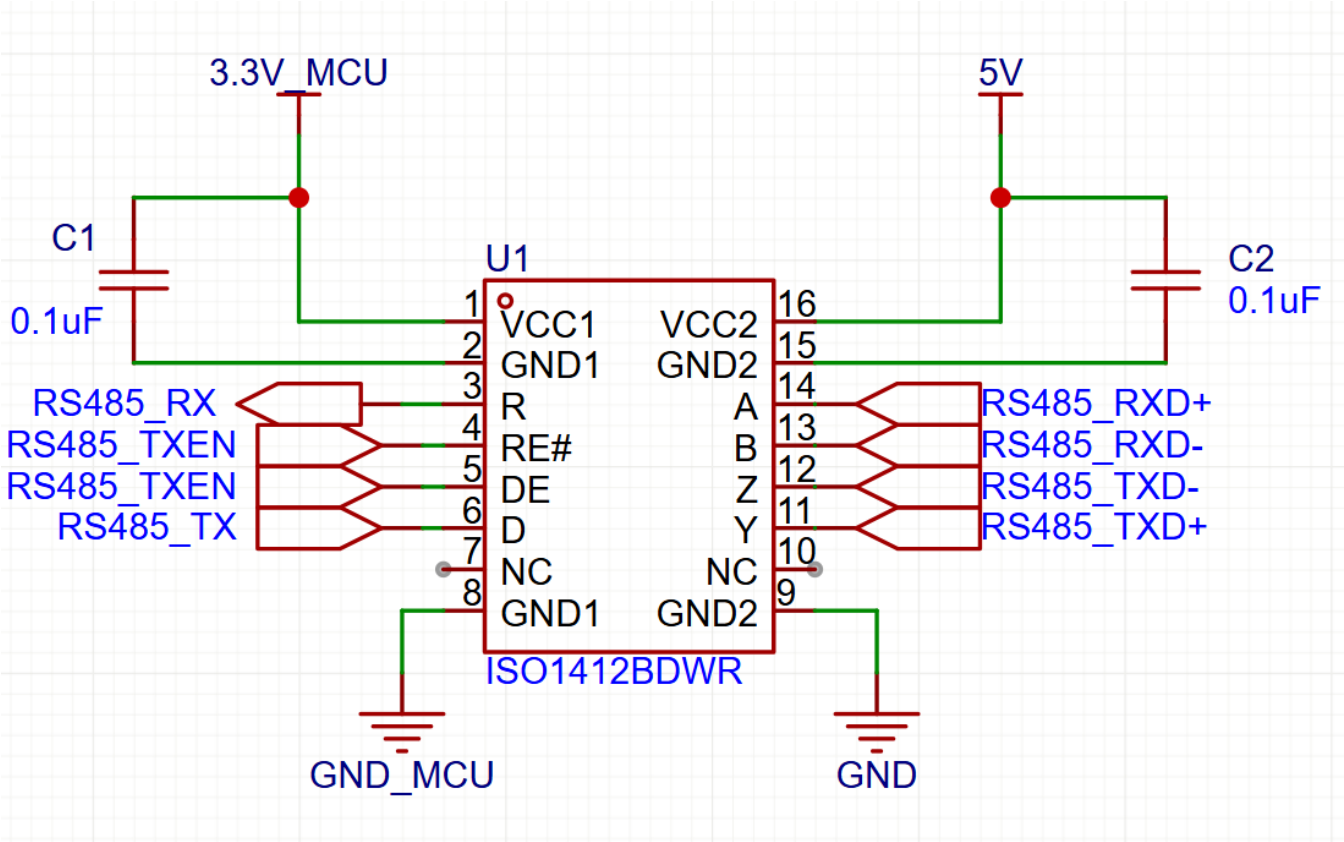
2.8. Communication Interface

2.8.1. Requirements

The communication interface must support reliable data exchange in a noisy motor-control environment.
RS-485 is selected due to its differential signaling and high noise immunity. Electrical isolation is required to protect the MCU from ground loops and surge voltages.

2.8.2. Hardware Overview

An isolated RS-485 transceiver is used to separate the MCU domain from the field communication bus.
The transceiver operates with independent power supplies on each side, ensuring galvanic isolation.



2.8.3. Component Description

- Isolated RS-485 transceiver: Converts UART signals to differential RS-485 signals and provides isolation.
- Decoupling capacitors (0.1 µF): Placed close to each supply pin to stabilize operation, as recommended by the datasheet.

2.8.4. Operating Principle

The MCU transmits and receives data via UART. The isolated transceiver converts logic-level signals into differential RS-485 signals while maintaining electrical isolation between the MCU and the communication bus.

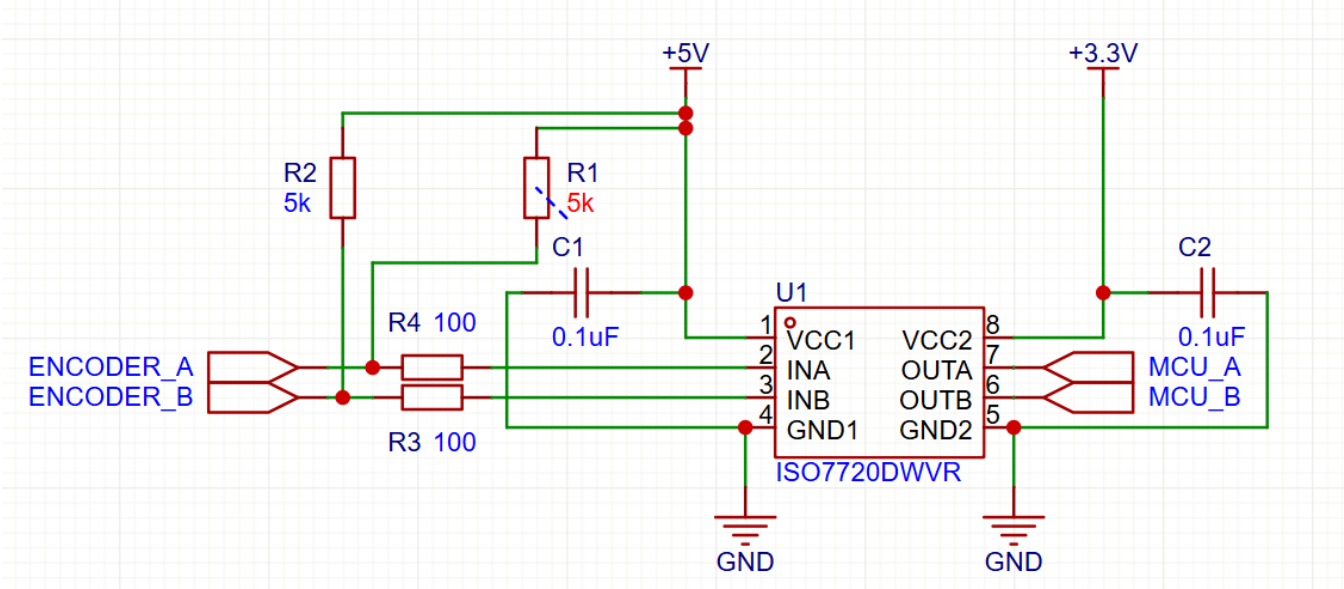
2.9. Encoder Interface

2.9.1. Requirements

The system requires accurate speed and direction feedback from an incremental encoder. The encoder interface must be isolated and resistant to electrical noise generated by the motor.

2.9.2. Hardware Overview

The encoder interface is implemented using the ISO7720 dual-channel digital isolator, separating the encoder side (5 V) from the MCU side (3.3 V).



2.9.3. Component Description

- Incremental encoder (A, B): Provides quadrature signals for speed and direction detection.
- Pull-up resistors R3, R4 (5 kΩ): Required because the encoder outputs are open-collector.

$$I \approx \frac{5V}{5k\Omega} \approx 1mA$$

This current ensures fast edges while remaining within safe limits.

- Series resistors R1, R2 (100 Ω): Improve signal integrity and reduce ringing without affecting pulse timing.
- ISO7720 digital isolator: Transfers logic states across the isolation barrier with high CMTI.
- Decoupling capacitors C1, C2 (0.1 μF): Placed at each VCC pin as required by the datasheet.

2.9.4. Operating Principle

The pull-up resistors generate logic HIGH levels. When the encoder output transistor turns ON, the signal is pulled LOW with respect to GND1. The ISO7720 isolates and transfers the logic states to the MCU for quadrature decoding.

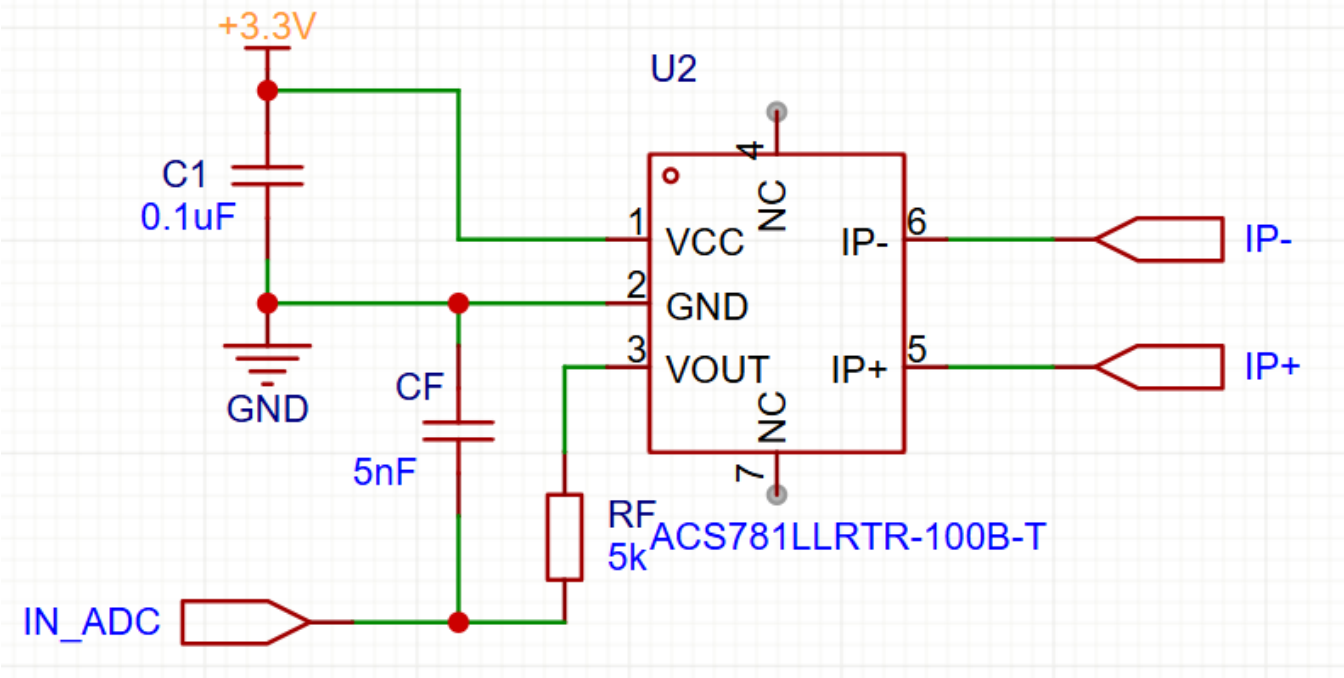
2.10. Current Sensor

2.10.1. Requirements

Motor current must be measured accurately and safely without exposing the MCU to high-power signals.

2.10.2. Hardware Overview

A Hall-effect current sensor (ACS781) is used to provide isolated current measurement.



2.10.3. Component Description

- ACS781: Measures motor current using Hall-effect sensing and provides inherent galvanic isolation.
- RC output filter: Reduces high-frequency switching noise before ADC sampling.
- Decoupling capacitor (0.1 µF): Stabilizes the sensor supply, as recommended by the datasheet.

2.10.4. Operating Principle

The current flowing through the sensor generates a proportional output voltage, which is filtered and sampled by the MCU ADC.

2.11. User Interface

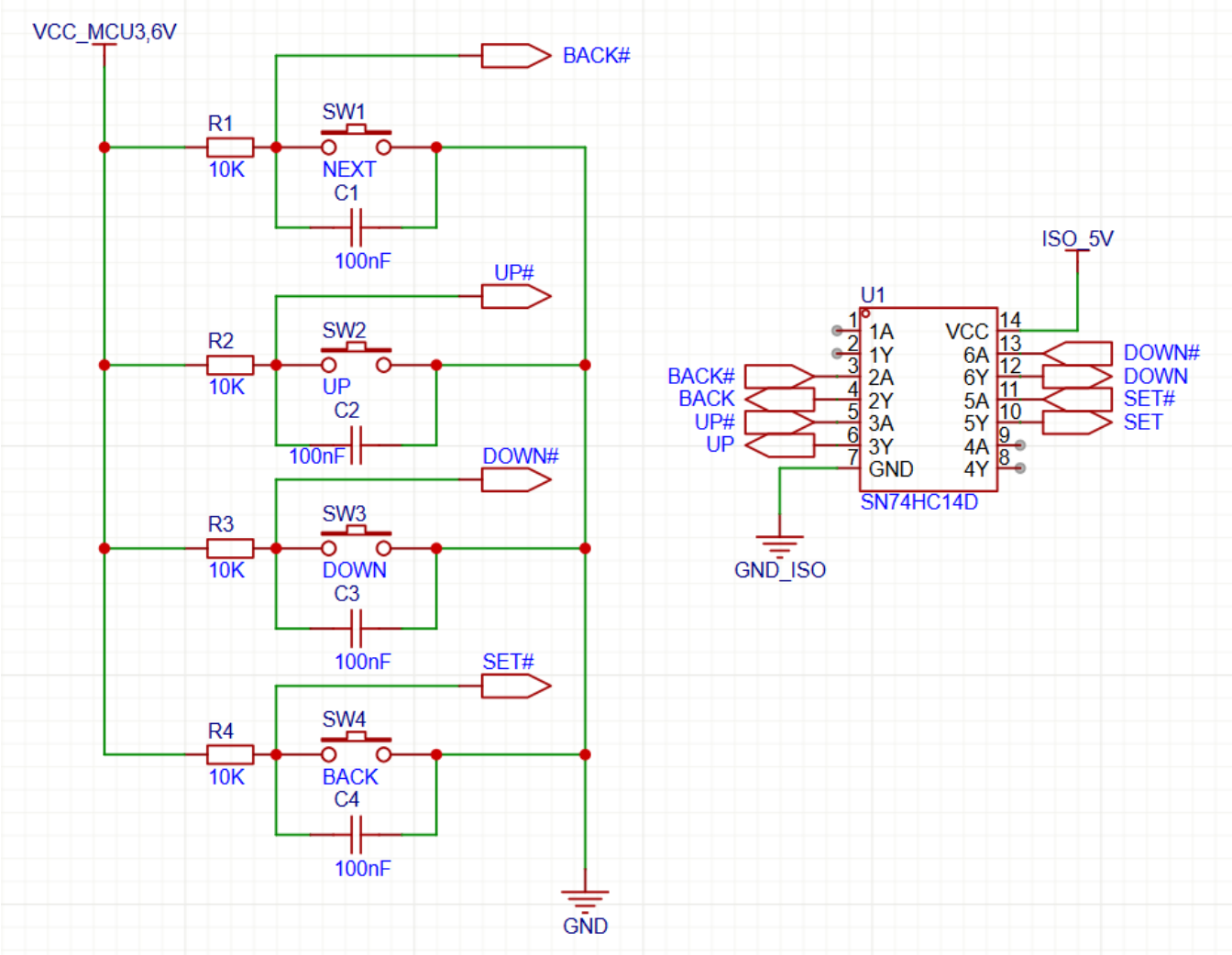
2.11.1. Requirements

The user interface must allow reliable user interaction for parameter setting and system monitoring.
Input signals must be immune to noise and mechanical contact bounce, while output information must be clearly displayed.

2.11.2. Button Interface

2.11.2.1. Hardware Overview

The button interface consists of multiple push buttons used for menu navigation and parameter adjustment.
Each button input is connected to the MCU through signal conditioning circuitry to ensure stable logic levels.



2.11.2.2. Component Description

- Push buttons: Provide user commands such as UP, DOWN, SET, and BACK.
- Pull-up resistors: Define a default logic HIGH level and prevent floating inputs.
- RC debounce network: Suppresses short glitches caused by mechanical contact bounce.
- Schmitt-trigger inverter: Converts slow or noisy button signals into clean digital transitions with defined thresholds.

2.11.2.3. Operating Principle

When a button is pressed, the signal is pulled LOW and filtered by the RC network.
The Schmitt-trigger inverter reshapes the signal into a clean logic level, which is then read by the MCU GPIO or interrupt input.

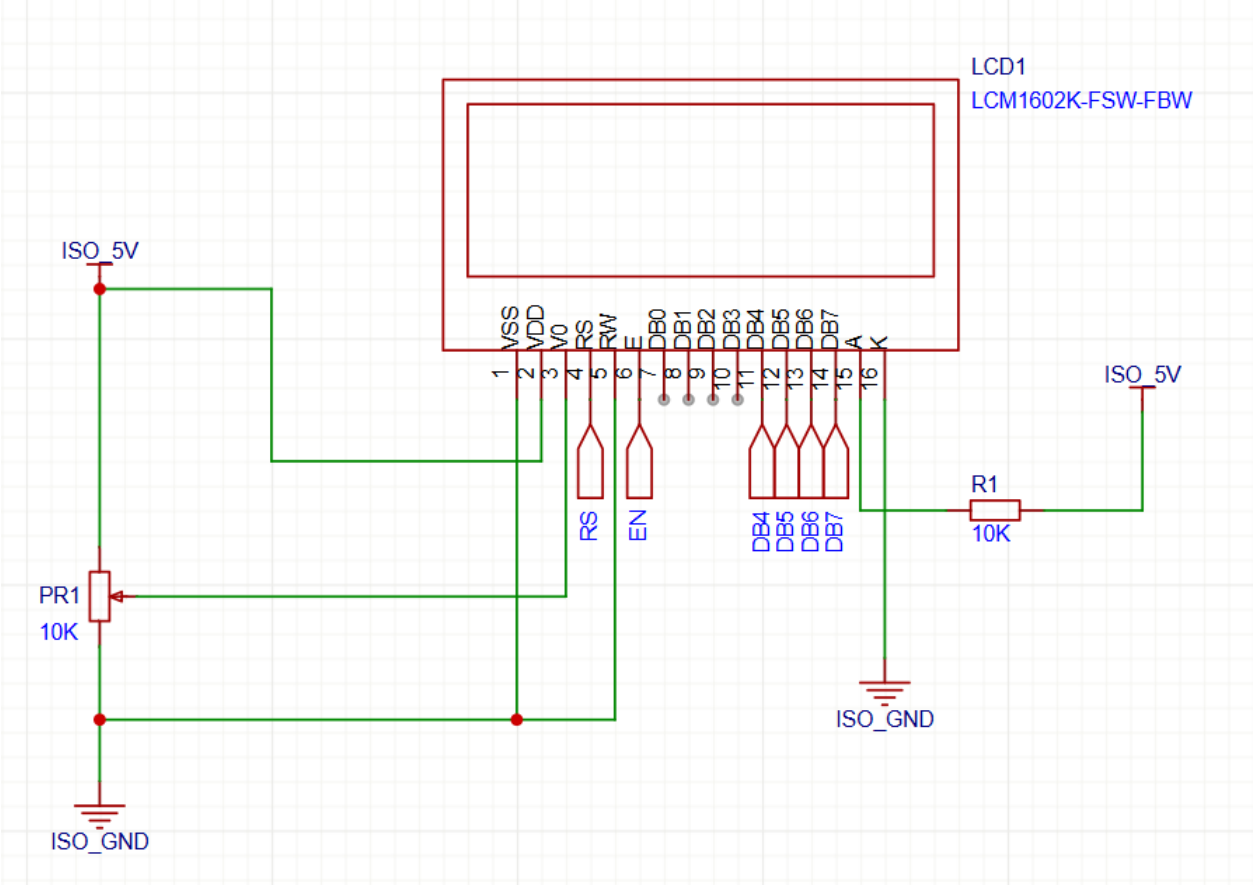
2.11.2.4. Summary

This hardware-based debouncing approach improves input reliability and reduces software processing requirements.

2.11.3. LCD Display Interface

2.11.3.1. Hardware Overview

LM016L (HD44780-compatible) character LCD is used to display operating parameters, system status, and menu information.
The LCD is interfaced with the MCU using a parallel data bus.



2.11.3.2. Component Description

- Character LCD: Provides text-based information display suitable for embedded systems.
- Control signals (RS, E): Manage command and data transfers between the MCU and LCD.
- 4-bit data interface: Reduces the number of MCU GPIO pins while maintaining full functionality.
- Contrast potentiometer: Allows adjustment of display contrast for optimal readability.
- Decoupling capacitor: Stabilizes the LCD supply voltage.

2.11.3.3. Operating Principle

The MCU sends commands and data to the LCD in 4-bit mode.
Display contents are updated periodically to reflect current system status and user-selected parameters.

2.12. MCU

- Requirement:

- Button: 4 pin (UP/DOWN/SET/BACK)
- LCD: 6 pin (RS, E, D4–D7)
- Digital input: 1 pin (DI)
- Analog input: 4 pin (SPI: SCK, MOSI, MISO, CS)
- Digital output: 1 pin (DO)
- Analog onput: 6 pin (SCLK, DIN(MOSI), DOUT(MISO), CS, LATCH, CLR)

- Timer:
    - Encoder: 2 pin (ENC_A, ENC_B)
    - PWM : 2 pin (PWM1, PWM2)
- Communication: 3 pin (TX, RX, DE)

=> We choose STM32F103C8T6 (LQFP48).

a. It meets the required pin count in a 48-pin package.

The system needs about 29 MCU pins for I/O and control.
STM32F103C8T6 in LQFP48 provides enough GPIO to cover this while still reserving critical pins for SWD debug, reset, boot, and power.

b. It has the exact peripherals the design depends on:

- 2× SPI: one dedicated for the external ADC, one for the DAC/output stage.
- UART for RS-485 communication.
- Timers with encoder interface for incremental encoder A/B counting.
- Multiple timers/channels for PWM, allowing the “2 PWM timers” requirement without bit-banging or heavy interrupts.

c. Good fit for a noisy motor-control environment

- reliable timers for PWM/encoder,
- robust digital I/O behavior,
- predictable real-time response.

d. Development ecosystem is mature and low risk

STM32F103C8T6 has:

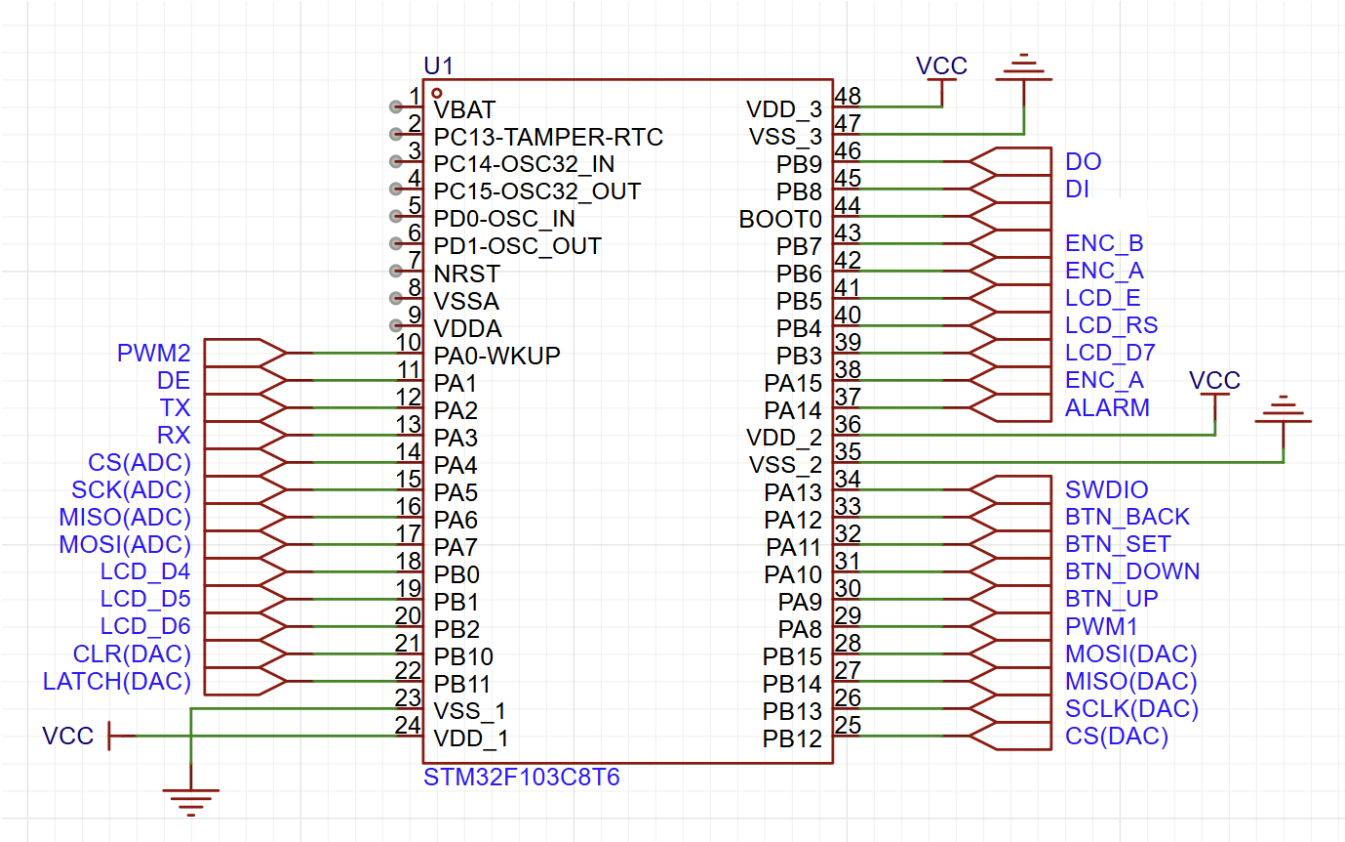
- huge community support (reference designs, examples, troubleshooting),
- stable toolchains (Keil/IAR/STM32CubeIDE),
- easy programming/debug via SWD,
- lots of existing libraries for LCD (HD44780), SPI ADC/DAC, RS-485 protocols, and encoder reading.

e. Cost, availability, and “good enough” performance

For this project, the MCU is not doing heavy DSP; it mainly coordinates I/O, communication, PWM, and timing. STM32F103C8T6 provides sufficient CPU performance and memory for:

- the UI (buttons + LCD),
- control logic (speed/torque mode selection),
- communication stack (RS-485),
- data handling for external ADC/DAC,
  without paying for a higher-end MCU that the design doesn’t actually need.

- Hardware design



3. Software Design

3.1 Task management

Task 1: Control Task (Control Loop)

Task 2: User Interface - LCD (Hierarchical State Machine)

Task 3: Communication Task (Event/Flag Driven)

Task 4: Safety Task (Continuous Monitoring)

Task	Priority	Logic Model
Safety Task	Highest	Flag Driven / Continuous Monitoring
Control Task	High	State Machine (Normal/Error)
Comm Task	Medium	Master-Slave (TX/RX)
UI Task	Low	Hierarchical State Machine

3.2 Code Program

```
// Priority Definitions (Higher number = Higher priority)

#define PRIORITY_SAFETY   4  // Highest: Must monitor current continuously

#define PRIORITY_CONTROL  3  // High: Must execute PID loop and handle safe states [cite: 2, 8]

#define PRIORITY_COMM     2  // Medium: Handles master-slave data exchange [cite: 4, 32]
```

```
#define PRIORITY_UI       1  // Lowest: Manages LCD and user settings [cite: 3, 49]

// Task Handles
TaskHandle_t xSafetyHandle, xControlHandle, xCommHandle, xUIHandle;

void main(void) {
    // 1. Setup Hardware (GPIO, Sensors, LCD, UART)

    // 2. Create Tasks with specified priorities
    xTaskCreate(Task_Safety, "Safety", 128, NULL, PRIORITY_SAFETY, &xSafetyHandle);
    xTaskCreate(Task_Control, "Control", 256, NULL, PRIORITY_CONTROL,
&xControlHandle);
    xTaskCreate(Task_Comm, "Comm", 256, NULL, PRIORITY_COMM, &xCommHandle);
    xTaskCreate(Task_UI, "UI", 512, NULL, PRIORITY_UI, &xUIHandle);

    // 3. Start Scheduler
    vTaskStartScheduler();
}
```

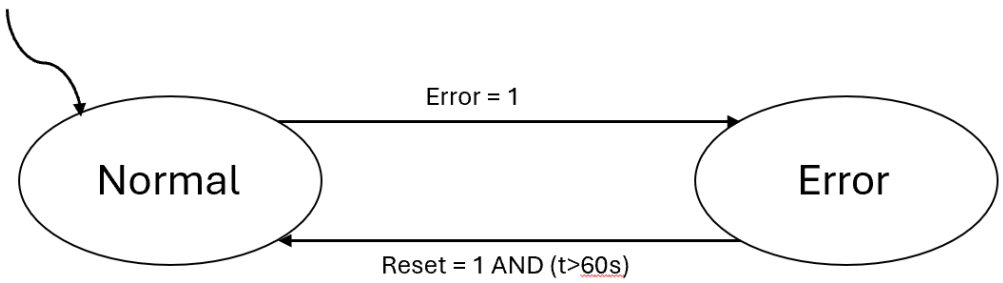
3.3 System FSM

Task 1: Control task

The Control Task controls the system during normal operation and moves the system into a safe state when a fault occurs. It consists of two states: Normal and Error.

- Normal:
  Runs the control operation.
  → Switches to Error when Error = 1.

- Error:
  Stops/holds operation in a safe fault state.
  → Returns to Normal only when Reset = 1 and the system has waited t > 60 s.

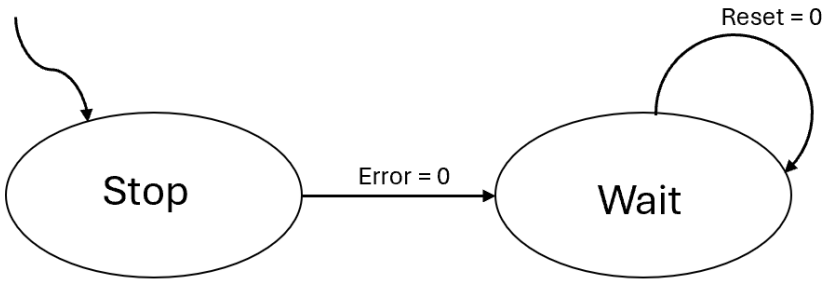


➢ ***Error:***

The Error state handles fault conditions by stopping the system and only allowing recovery when the fault is cleared and a reset is requested. It consists of two sub-states: Stop and Wait.

- Stop:
  Immediately forces the system into a safe stop while the fault is active.
  → Switches to Wait when Error = 0 (fault cleared).

- Wait:
  Keeps the system disabled and waits for a user/system reset request.
  → Stays in Wait while Reset = 0.

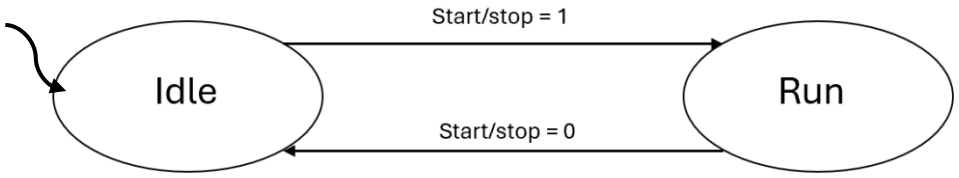


➢ ***Normal:***

The Normal state is the operating mode where the system performs its control function. It consists of two sub-states: Idle and Run.

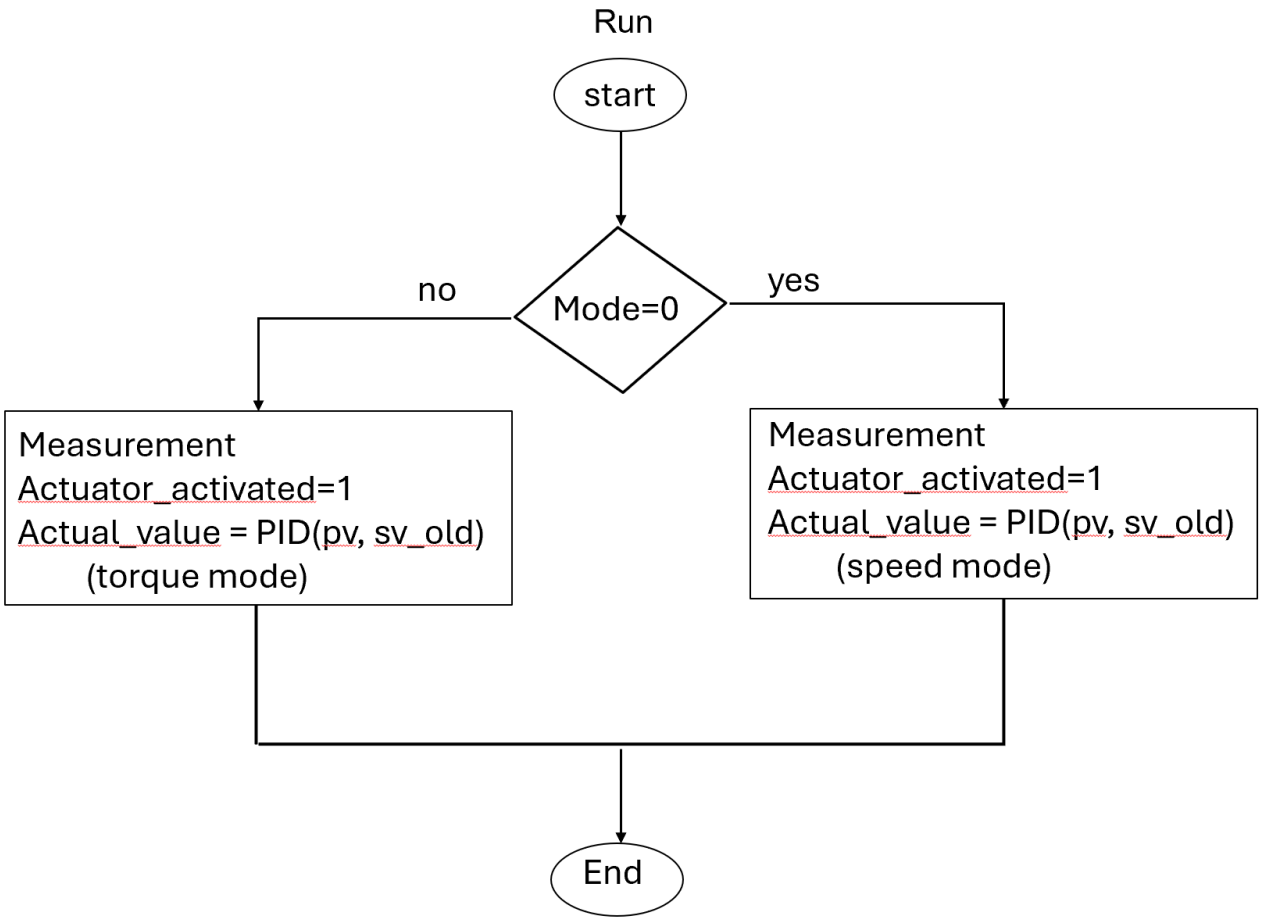
- Idle:
  The controller is in standby (no active control output).
  → Switches to Run when Start/Stop = 1.

- Run:
  The controller is active and executes the control operation.
  → Returns to Idle when Start/Stop = 0.



➢ **Action function of Run:**

- First it needs to choose what mode the system will operate with mode=0 is speed control and mode=1 is torque control.



- This action actually consists of sequential three main sub-process: Measurement, PID calculation, Actuator. The output of the preceding state serves as the mandatory input for the succeeding state.

Code program:

```
void Task_Control(void *pvParameters) {
  while(1) {
    if (Error == 1) {
      // Error State Logic
      if (fault_cleared == 1) { // Error = 0
        // Wait State: Wait for Reset and 60s timer
        if (Reset == 1 && timer_60s_done) {
          Error = 0;
        }
      } else {
        // Stop State: Force safe stop
        actuator_output(0);
      }
    } else {
      // Normal State Logic
```

```
        if (StartStop == 1) { // Run Sub-state
            pv = Measurement();
            Actual_value = PID_Calculation(pv, sv_old, mode); // mode 0=speed, 1=torque
            Actuator(Actual_value);
        } else { // Idle Sub-state
            Actuator(0); // Standby
        }
    }
    vTaskDelay(pdMS_TO_TICKS(50));
  }
}
```

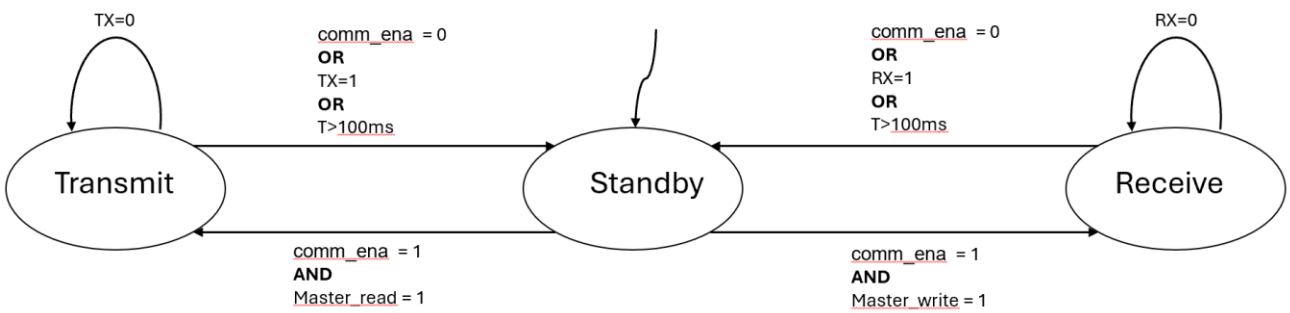
Task 2: Communication

The Communication Task handles data exchange using a master–slave method. In this system, the controller acts as the slave, while the PC or higher-level device acts as the master. The task consists of three states: Standby, Transmit, and Receive.

- Standby:
  This is the default state when the task starts. The slave waits for communication to be enabled and for a request from the master.
  → Go to Transmit when comm_ena = 1 and master_read = 1.
  → Go to Receive when comm_ena = 1 and master_write = 1.

- Transmit:
  The slave sends requested data to the master.
  → Stay in Transmit while TX = 0 (transmission not completed).
  → Return to Standby when comm_ena = 0, or TX = 1 (transmission completed), or T > 100 ms (timeout to prevent being stuck in Transmit).

- Receive:
  The slave receives a command/data packet from the master and updates/executes the requested action.
  → Stay in Receive while RX = 0 (reception not completed).
  → Return to Standby when comm_ena = 0, or RX = 1 (reception completed), or T > 100 ms (timeout to prevent being stuck in Receive).



Code program:

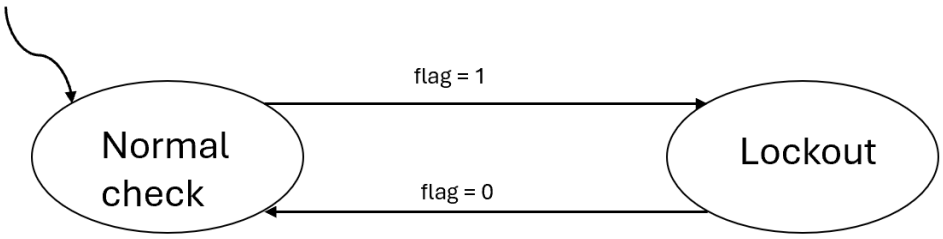
```
void Task_Communication(void *pvParameters) {
    while(1) {
        if (comm_ena == 1) {
            switch(comm_state) {
                case STANDBY:
                    if (master_read == 1) comm_state = TRANSMIT;
                    else if (master_write == 1) comm_state = RECEIVE;
                    break;

                case TRANSMIT:
                    send_data();
                    if (TX == 1 || timeout > 100) comm_state = STANDBY;
                    break;

                case RECEIVE:
                    get_data();
                    if (RX == 1 || timeout > 100) comm_state = STANDBY;
                    break;
            }
        }
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

Task 3: Safety

The Safety Task monitors the motor current and triggers a system fault when the current exceeds a safe limit. It consists of two states: Normal Check and Lockout.

- Normal Check:
  The task reads the current from the sensor and compares it with the threshold.
  → If the current is higher than the threshold, set flag = 1 and transition to Lockout.

- Lockout:
  The system enters a protection mode and sets Error = 1, which forces the Control Task to move to its Error state.
  → When the fault condition is cleared (flag = 0), return to Normal Check and continue monitoring.



Code program:

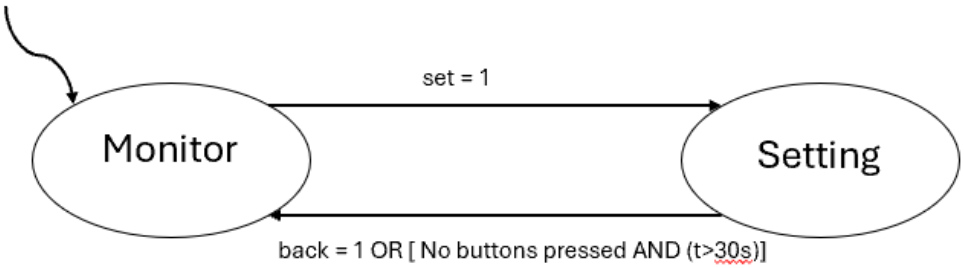
```
void Task_Safety(void *pvParameters) {
    while(1) {
        switch(safety_state) {
            case NORMAL_CHECK:
                current = read_sensor();
                if (current > threshold) {
                    flag = 1;
                    safety_state = LOCKOUT;
                }
                break;

            case LOCKOUT:
                Error = 1; // Forces Control Task to Error state
                if (flag == 0) {
                    safety_state = NORMAL_CHECK;
                }
                break;
        }
        vTaskDelay(pdMS_TO_TICKS(10)); // High frequency monitoring
    }
}
```

Task 4: UI (user interface)

The UI Task lets the user monitor system status and configure operating parameters using buttons and the display. It consists of two states: Monitor and Setting.

- Monitor:
  This is the default state when the UI task starts. It displays key parameters (e.g., setpoint, process value, flag, operating mode, etc.).
  → Switches to Setting when set = 1 (user presses the Setting button).

- Setting:
  The user adjusts system parameters such as setpoint, mode, and communication/connection enable.
  → Returns to Monitor when back = 1, or when no button is pressed for 30 s (timeout).



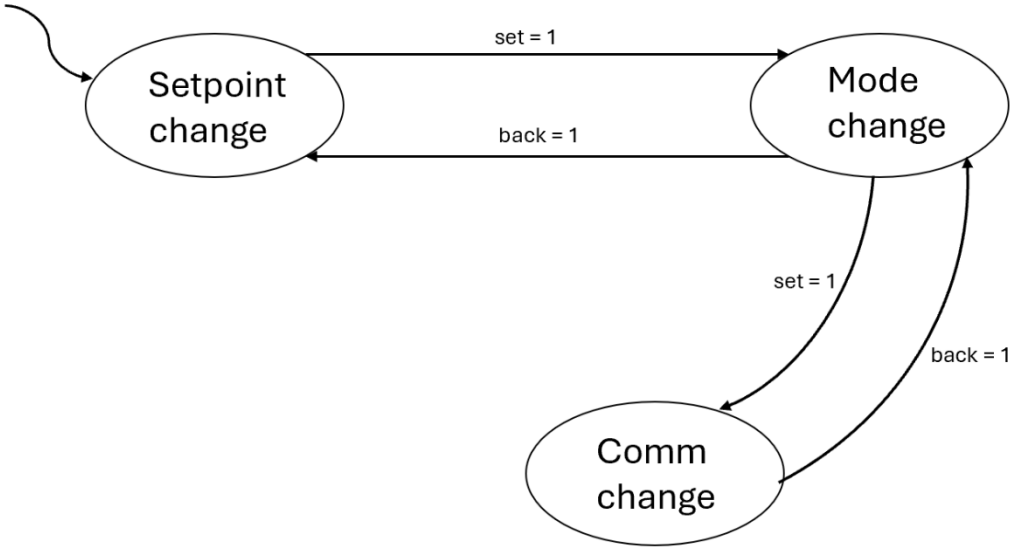
➢ **Setting:**

The Setting state is a configuration menu that lets the user adjust setpoint, mode, and communication settings. It consists of three sub-states: Setpoint Change, Mode Change, and Comm Change.

- Setpoint Change:
  The first menu screen where the user changes the setpoint value.
  → Go to Mode Change when set = 1.

- Mode Change:
  The user selects or changes the operating mode.
  → Go back to Setpoint Change when back = 1.
  → Go to Comm Change when set = 1.

- Comm Change:
  The user enables/disables communication or connection settings.
  → Return to Mode Change when back = 1.



➢ **Action function of setpoint change, mode change, comm change**

The Setpoint Change screen allows the user to adjust the setpoint value with different step sizes depending on how long the Up/Down button is held. It consists of three sub-states: Change, Fast_10, and Fast_20.

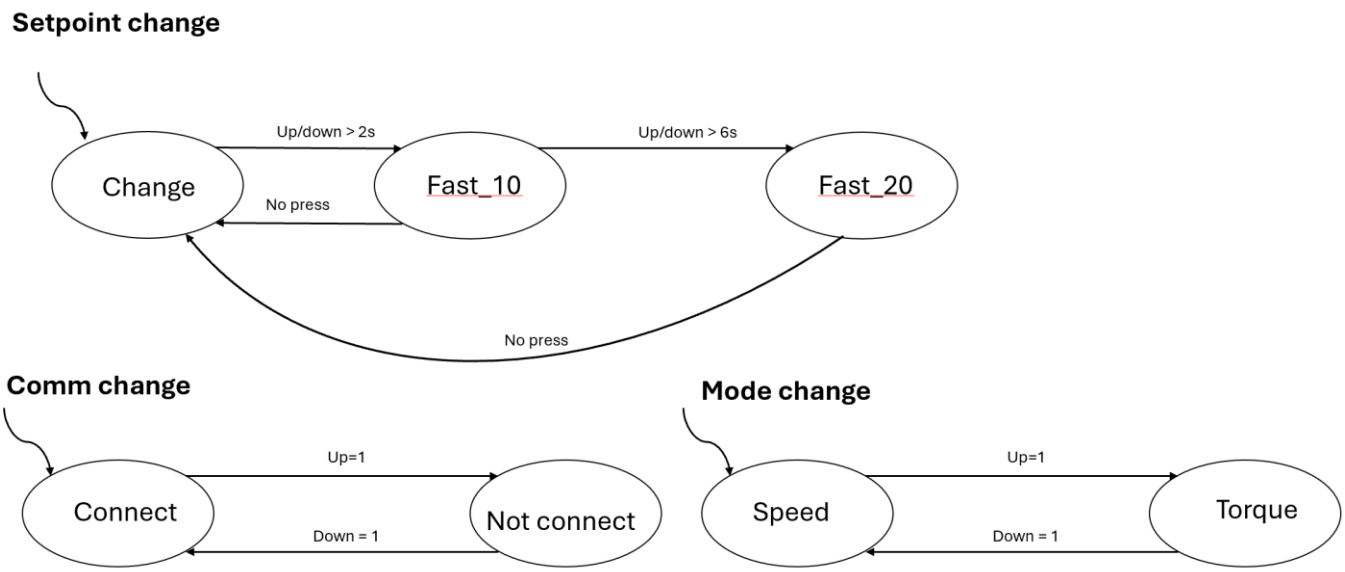
- Change:
  Normal adjustment mode. Each Up/Down press changes the setpoint by **1 unit**.
  → If Up/Down is held > 2 s, switch to Fast_10.
- Fast_10:
  Fast adjustment mode. Each Up/Down action changes the setpoint by 10 units.
  → If Up/Down is held > 6 s, switch to Fast_20.
  → If the button is released (no button pressed), return to Change.
- Fast_20:
  Very fast adjustment mode. Each Up/Down action changes the setpoint by 20 units.
  → If the button is released (no button pressed), return to Change.

Mode change screen lets the user select the operating mode. It consists of two sub-states: Speed and Torque.

- Speed:
  The system is set to Speed control mode.
  → Switches to Torque when Up = 1.
- Torque:
  The system is set to Torque control mode.
  → Returns to Speed when Down = 1.

Comm change screen lets the user enable or disable communication with an external master device. It consists of two sub-states: Connect and Not connect.

- Connect:
  Communication is enabled (the slave is allowed to communicate with the master).
  → Switches to Not connect when Up = 1.
- Not Connect:
  Communication is disabled.
  → Returns to Connect when Down = 1.



Code program:

```
void Task_UI(void *pvParameters) {
```

```
    while(1) {
        if (ui_state == MONITOR) {
            display_status(setpoint, pv, flag, mode);
            if (set_button == 1) ui_state = SETTING;
        }
        else if (ui_state == SETTING) {
            handle_sub_menus(); // Setpoint Change, Mode Change, Comm Change
            if (back_button == 1 || inactivity_timer > 30000) ui_state = MONITOR;
        }

        // Accelerated Setpoint Logic
        if (hold_time > 6000) setpoint += 20;      // Fast_20
        else if (hold_time > 2000) setpoint += 10; // Fast_10
        else if (press) setpoint += 1;             // Change

        vTaskDelay(pdMS_TO_TICKS(200));
    }
}
```